# 알 고 리 즘

## Brute Force Algorithm

# 알고리즘 전략

1. Brute force
2. Divide and conquer
3. Decrease and conquer
4. Transform and conquer
5. Greedy Method
6. Dynamic Programming
7. Backtracking
8. Branch and Bound

# Brute Force Algorithm 개념

# 누가 이길까요?

# Brute Force

- A straightforward approach, usually based directly on the problem's statement and definitions of the concepts involved

- 예:
  - Computing $a^n$ ($a > 0$, n a nonnegative integer)
  - Computing n!
  - Multiplying two matrices
  - Searching for a key of a given value in a list

# Concepts of Brute Force Algorithm

- A brute-force algorithm solves a problem in the most **simple**, direct or obvious way.
- As a result, such an algorithm can end up doing far **more work** to solve a given problem than a more clever or sophisticated algorithm might do.
- On the other hand, a brute-force algorithm is often **easier to implement** than a more sophisticated one and, because of this simplicity, sometimes it can be more efficient.
- Typically, a brute-force algorithm solves such a problem by **exhaustively enumerating all the possibilities**. i.e., for every decision we consider each possible outcome.

# The Simplest Approach

- Brute Force - the simplest of the design strategies
  - Is a straightforward approach to solving a problem, usually directly based on the problem's statement and definitions of the concepts involved.
  - **Just do it** - the brute-force strategy is easiest to apply.
  - Results in an algorithm that can be improved with a modest amount of time.
- Brute force is important due to its wide applicability and simplicity.

# Examples of Brute Force Algorithm

- Selection Sort
- Bubble Sort
- String Matching
- Sequential Search
- Exhaustive Search
  - Traveling Salesman Problem (Hamilton Circuits)
  - Knapsack Problem
  - Job Assignment Problem

# Selection Sort

# Selection Sort (선택 정렬)

- 정렬 대상 자료의 값 중에서 가장 작은 값을 검색
- 맨 앞의 자리에 최소값을 옮기고 원래 있었던 값은 최소값의 원래 자리에 배치
- 이제 전체 정렬 대상 중 맨 앞의 자료 1개 정렬 완료!
- 맨 앞의 자료를 제외한 n-1개의 자료를 스캔하여 n-1개의 자료 중 가장 작은 최소값을 검색
- 두 번째로 작은 값을 두 번째 자리에 배치시키고 역시 원래 있던 값을 두 번째 작은 값이 있던 자리로 이동
- 이제 전체 정렬 대상 중 맨 앞의 자료 2개 정렬 완료!
- 위의 방법을 반복하여 전체 자료 정렬 완성

# Selection Sort – 정렬 과정

| 89 , 45 , 68 , 90 , 29 , 34 , **17**

17 | 45 , 68 , 90 , **29** , 34 , 89

17 , 29 | 68 , 90 , 45 , **34** , 89

17 , 29 , 34 | 90 , **45** , 68 , 89

17 , 29 , 34 , 45 | 90 , **68** , 89

17 , 29 , 34 , 45 , 68 | 90 , **89**

17 , 29 , 34 , 45 , 68 , 89 | 90

# Time Efficiency 분석

- $O(n^2)$

```
ALGORITHM  SelectionSort(A[0..n − 1])
    //Sorts a given array by selection sort
    //Input: An array A[0..n − 1] of orderable elements
    //Output: Array A[0..n − 1] sorted in ascending order
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min]   min ← j
        swap A[i] and A[min]
```

# Bubble Sort

# Bubble Sort (버블 정렬)

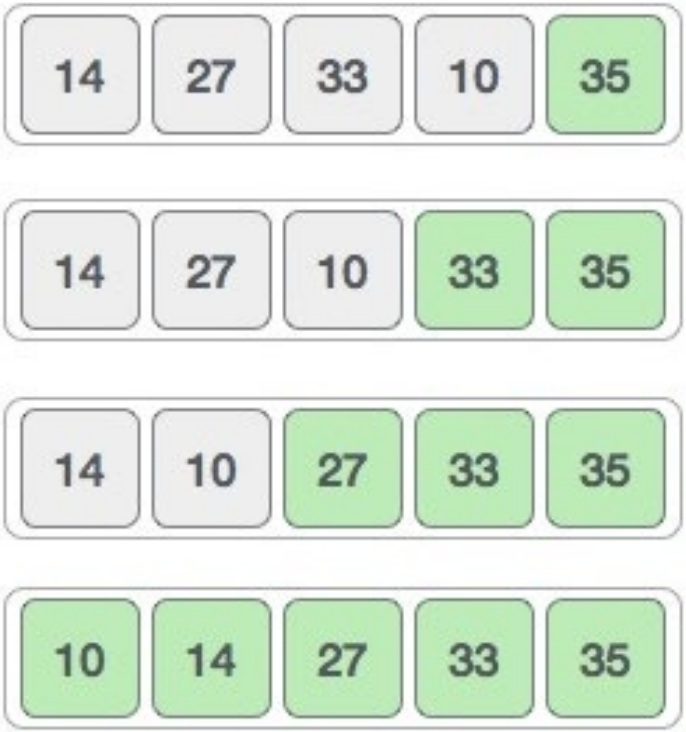
- 서로 인접한 두 원소를 검사하여 정렬
- 인접한 2개의 원소를 비교하여 크기가 순서대로 되어 있지 않으면 서로 위치를 교환
- 선택 정렬과 기본 개념이 유사
- 한 과정을 수행하고 나면 가장 큰 자료가 맨 뒤로 이동
- 그 다음 과정에서는 맨 끝에 있는 자료는 정렬에서 제외되고, 나머지를 수행하고 나면 끝에서 두 번째 자료까지 정렬
- 반복하여 전체를 정렬

# Bubble Sort – 정렬 과정

- 1개 정렬 완성을 위하여...

# Bubble Sort – 정렬 순서

- 최종 정렬 상태



# Time Efficiency

- $O(N^2)$

**Algorithm :** Bubble sort

```
Data: Input array A[]
Result: Sorted A[]
int i, j, k;
N = length(A);
for j = 1 to N do
    for i = 0 to N-1 do
        if A[i] > A[i+1] then
            temp = A[i];
            A[i] = A[i+1];
            A[i+1] = temp;
        end
    end
end
```

# String Matching

# String Matching

- Given a string of n characters called the **text** and a string of m characters (m<=n) called the **pattern,** find a substring of the text that matches the pattern.

- A brute force algorithm for string matching:
  - 맨 앞의 문자부터 패턴을 확인
  - 패턴 안에서 m개의 문자를 왼쪽부터 오른쪽으로 진행하며 비교
  - 서로 다른 문자가 나타나면, 패턴을 2번째 문자 부터 비교 시작 (shift one position to the right)
  - 패턴과 문자를 반복하여 비교
  - 같은 반복으로 문자열 전체를 비교

# String Matching – 비교 방법

```
N O B O D Y _ N O T I C E D _ H I M        Text

N O T                                      Pattern
  N
    N
      ......
            N O T      성공!
              N
```

# Time Efficiency

- O(m·n) m: pattern 길이 & n: text 길이

```
ALGORITHM  BruteForceStringMatch(T[0..n − 1], P[0..m − 1])
    //Implements brute-force string matching
    //Input: An array T[0..n − 1] of n characters representing a text and
    //        an array P[0..m − 1] of m characters representing a pattern
    //Output: The index of the first character in the text that starts a
    //        matching substring or −1 if the search is unsuccessful
    for i ← 0 to n − m do
        j ← 0
        while j < m and P[j] = T[i + j] do
            j ← j + 1
        if j = m return i
    return −1
```

# Sequential Search

# Sequential Search (순차 탐색)

- 처음부터 끝까지 차례대로 원소 비교하며 탐색

- 데이터를 따로 조작할 필요가 없어 단순하지만 비효율적
- 정렬되어 있지 않은 데이터에서의 탐색
- 단방향으로 탐색을 수행하기 때문에 선형 탐색(Linear Search) 이라고도 함
- 탐색에 실패한 경우 n+1 또는 -1을 반환

# Sequential Search (순차 탐색) 예

Given array – 12  24  35  42  46  49  50  82  99

Target value – 52

Return value : 10 (where n = 9) 즉, 탐색 실패

Given array – 12  24  35  42  46  49  50  82  99

Target value – 42

Return value : 4

# Time Efficiency

- O(n)

```
linearSearch(A,n,t)
  i = 0
  while (i < n) and (A[i] != t)
    i = i + 1
  return i
```

  - 평균적으로 (n+1)/2번의 비교를 거치며, 최악의 경우 n번의 비교

# Exhaustive Search

# Exhaustive Search (완전 탐색)

- Exhaustive search is a brute-force approach to combinatorial problems.
  - It suggests generating each and every combinatorial object of the problem, selecting those of them of that satisfy the problem's constraints and then finding a desired object.
  - Impractical for all but applicable to very small instances of problems.
- Examples:
  - Traveling salesman problem
    - Finding the shortest tour through a given set of *n* cities that visits each city exactly once before returning to the city where it started.
  - Knapsack problem
    - Finding the most valuable list of out-of *n* items that fit into the knapsack.
  - Job Assignment problem
    - Finding an assignment of n people to execute *n* jobs with the smallest total cost.

# Traveling Salesman Problem

# Traveling Salesman Problem

- Given : A complete, weighted graph
- Find : A Hamiltonian circuit of minimum weight

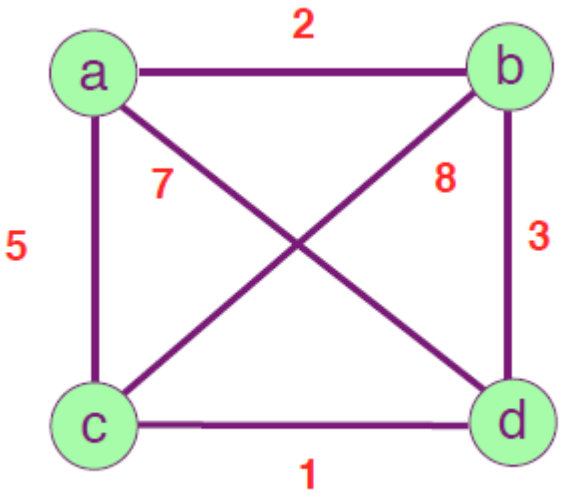
※ A circuit that starts at a vertex of a graph, passes through every vertex exactly once, and returns to the starting vertex is called a HAMILTONIAN CIRCUIT.

  - named for William Rowan Hamilton who studied them in the 1800's.

- Algorithm
  - List all Hamiltonian circuits
  - Compute the sum of weights (total weight) for each circuit
  - Choose the one with the smallest total weight

# TSP – how to work

| Tour | Length |
|---|---|
| a b c d a | 2+8+1+7 = 18 |
| a b d c a | 2+3+1+5 = **11** |
| a c b d a | 23 |
| a c d b a | **11** |
| a d b c a | 23 |
| a d c b a | 18 |




More than one optimal solutions.

-same result for the reverse order

# Hamiltonian Circuits

***Step 1:*** Choose a starting point.

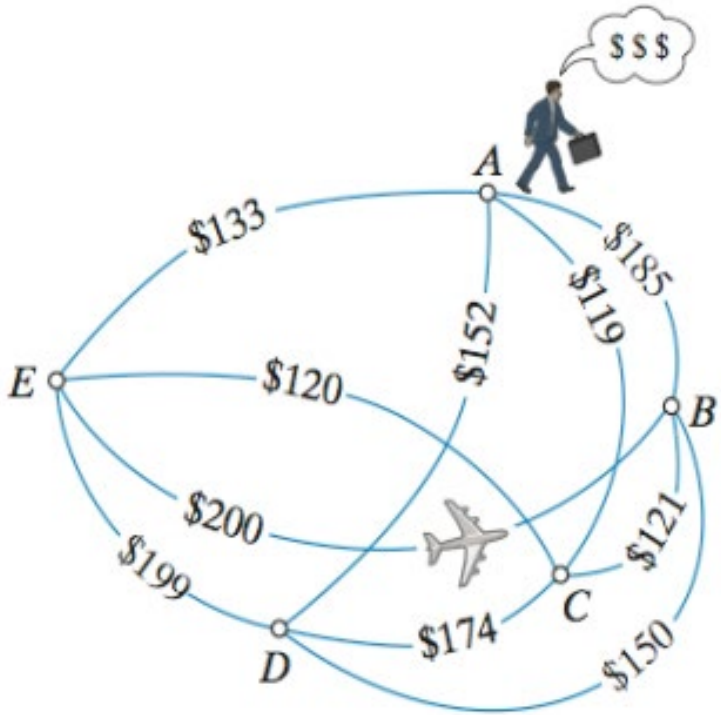
***Step 2:*** List all the Hamiltonian circuits with that starting point.

***Step 3:*** Find the total weight of each circuit.

***Step 4:*** Choose a Hamiltonian circuit with the smallest total weight.

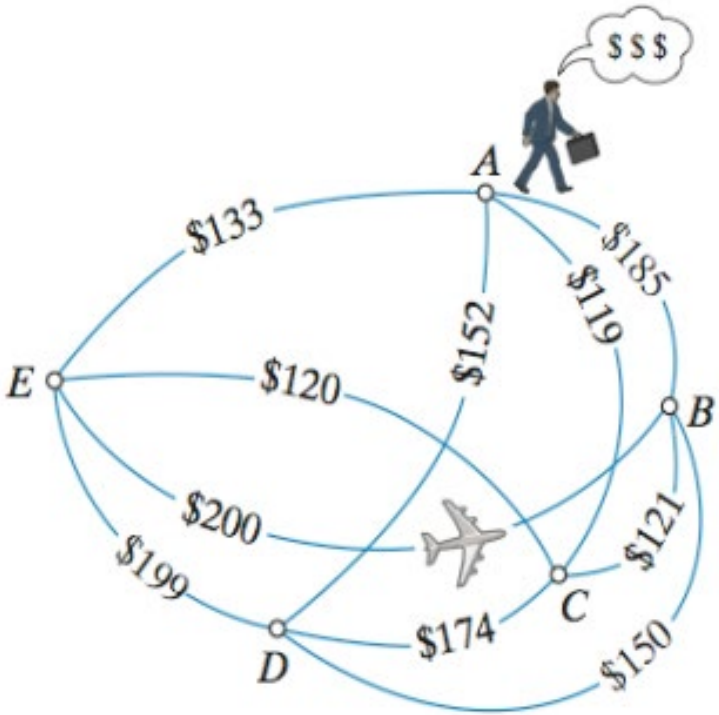
# Optimal Hamiltonian Circuits

- 항상 최선의 답을 찾으므로 Brute Force 방식으로 해결하는 경우 Optimal Hamiltonian Circuits이라고 함
- 다음의 문제에서 Optimal Hamiltonian Circuits 찾아보세요.



# Optimal Hamiltonian Circuits 예

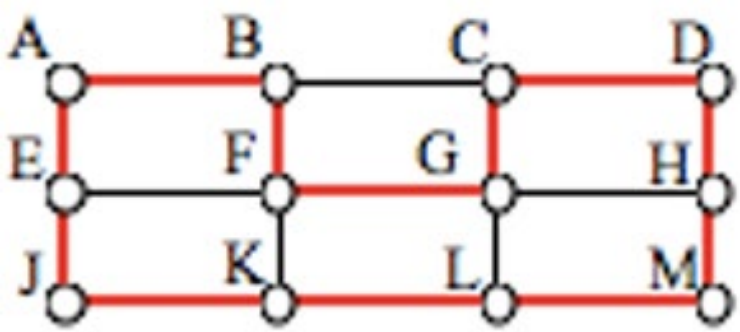
- ADBCEA / AECBDA




| | Hamilton circuit | Total cost | Mirror-image circuit |
|---|---|---|---|
| 1 | $A, B, C, D, E, A$ | 185 + 121 + 174 + 199 + 133 = 812 | $A, E, D, C, B, A$ |
| 2 | $A, B, C, E, D, A$ | 185 + 121 + 120 + 199 + 152 = 777 | $A, D, E, C, B, A$ |
| 3 | $A, B, D, C, E, A$ | 185 + 150 + 174 + 120 + 133 = 762 | $A, E, C, D, B, A$ |
| 4 | $A, B, D, E, C, A$ | 185 + 150 + 199 + 120 + 119 = 773 | $A, C, E, D, B, A$ |
| 5 | $A, B, E, C, D, A$ | 185 + 200 + 120 + 174 + 152 = 831 | $A, D, C, E, B, A$ |
| 6 | $A, B, E, D, C, A$ | 185 + 200 + 199 + 174 + 119 = 877 | $A, C, D, E, B, A$ |
| 7 | $A, C, B, D, E, A$ | 119 + 121 + 150 + 199 + 133 = 722 | $A, E, D, B, C, A$ |
| 8 | $A, C, B, E, D, A$ | 119 + 121 + 200 + 199 + 152 = 791 | $A, D, E, B, C, A$ |
| 9 | $A, C, D, B, E, A$ | 119 + 174 + 150 + 200 + 133 = 776 | $A, E, B, D, C, A$ |
| 10 | $A, C, E, B, D, A$ | 119 + 120 + 200 + 150 + 152 = 741 | $A, D, B, E, C, A$ |
| 11 | $A, D, B, C, E, A$ | 152 + 150 + 121 + 120 + 133 = 676 | $A, E, C, B, D, A$ |
| 12 | $A, D, C, B, E, A$ | 152 + 174 + 121 + 200 + 133 = 780 | $A, E, B, C, D, A$ |

# Hamiltonian Circuit의 이해

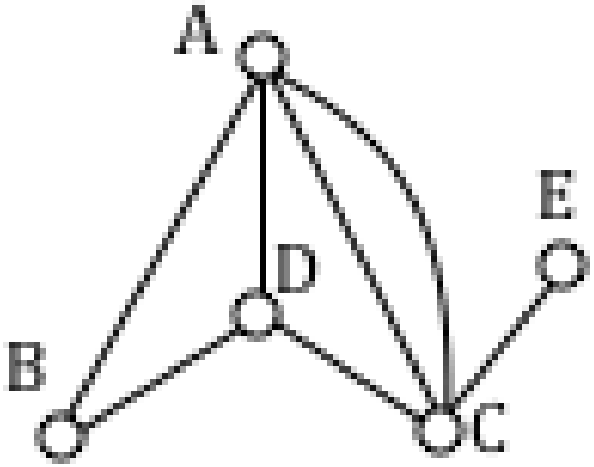
- 모든 vertex를 방문하여 시작 지점으로 돌아오기
  - 모든 edge를 거칠 필요는 없음




- ABFGCDHMLKJEA
- AEJKLMHDCGFBA

# 【참고】 Hamiltonian Path

- visits every vertex once with no repeats, but does not have to start and end at the same vertex
  - 모든 vertex를 방문
  - 시작과 끝이 같지 않아도 됨
- 아래의 그래프는 Hamiltonian Circuit을 만족하는가?



# Brute Force 접근 방식의 장단점

- 장점
  - Guaranteed to find the most efficient circuit
- 단점
  - Can be a lot of work to carry out the algorithm

# 효율성

- 비효율적 해결법
  - Inefficient algorithm
    - an algorithm for which the number of steps needed to carry it out grows disproportionally with the size of the problem

# Graph 자료

## Data Structure

```
def create_data_model():
    """Stores the data for the problem."""
    data = {}
    data['distance_matrix'] = [
        [0, 2451, 713, 1018, 1631, 1374, 2408, 213, 2571, 875, 1420, 2145, 1972],
        [2451, 0, 1745, 1524, 831, 1240, 959, 2596, 403, 1589, 1374, 357, 579],
        [713, 1745, 0, 355, 920, 803, 1737, 851, 1858, 262, 940, 1453, 1260],
        [1018, 1524, 355, 0, 700, 862, 1395, 1123, 1584, 466, 1056, 1280, 987],
        [1631, 831, 920, 700, 0, 663, 1021, 1769, 949, 796, 879, 586, 371],
        [1374, 1240, 803, 862, 663, 0, 1681, 1551, 1765, 547, 225, 887, 999],
        [2408, 959, 1737, 1395, 1021, 1681, 0, 2493, 678, 1724, 1891, 1114, 701],
        [213, 2596, 851, 1123, 1769, 1551, 2493, 0, 2699, 1038, 1605, 2300, 2099],
        [2571, 403, 1858, 1584, 949, 1765, 678, 2699, 0, 1744, 1645, 653, 600],
        [875, 1589, 262, 466, 796, 547, 1724, 1038, 1744, 0, 679, 1272, 1162],
        [1420, 1374, 940, 1056, 879, 225, 1891, 1605, 1645, 679, 0, 1017, 1200],
        [2145, 357, 1453, 1280, 586, 887, 1114, 2300, 653, 1272, 1017, 0, 504],
        [1972, 579, 1260, 987, 371, 999, 701, 2099, 600, 1162, 1200, 504, 0],
    ]
    return data

def main():
    data = create_data_model()

if __name__ == '__main__':
    main()
```

array indices correspond to the locations in the following order:

0. New York - 1. Los Angeles –
2. Chicago - 3. Minneapolis –
4. Denver - 5. Dallas –
6. Seattle - 7. Boston –
8. San Francisco - 9. St. Louis –
10. Houston - 11. Phoenix –
12. Salt Lake City

```
{'distance_matrix': [[0, 2451, 713, 1018, 1631, 1374, 2408, 213, 2571, 875, 1420, 2145, 1972], [2451, 0, 1745, 1524, 831, 1240, 959, 2596, 403, 1589, 1374, 357, 579], [713, 1745, 0, 355, 920, 803, 1737, 851, 1858, 262, 940, 1453, 1260], [1018, 1524, 355, 0, 700, 862, 1395, 1123, 1584, 466, 1056, 1280, 987], [1631, 831, 920, 700, 0, 663, 1021, 1769, 949, 796, 879, 586, 371], [1374, 1240, 803, 862, 663, 0, 1681, 1551, 1765, 547, 225, 887, 999], [2408, 959, 1737, 1395, 1021, 1681, 0, 2493, 678, 1724, 1891, 1114, 701], [213, 2596, 851, 1123, 1769, 1551, 2493, 0, 2699, 1038, 1605, 2300, 2099], [2571, 403, 1858, 1584, 949, 1765, 678, 2699, 0, 1744, 1645, 653, 600], [875, 1589, 262, 466, 796, 547, 1724, 1038, 1744, 0, 679, 1272, 1162], [1420, 1374, 940, 1056, 879, 225, 1891, 1605, 1645, 679, 0, 1017, 1200], [2145, 357, 1453, 1280, 586, 887, 1114, 2300, 653, 1272, 1017, 0, 504], [1972, 579, 1260, 987, 371, 999, 701, 2099, 600, 1162, 1200, 504, 0]]}
```

# Knapsack Problem

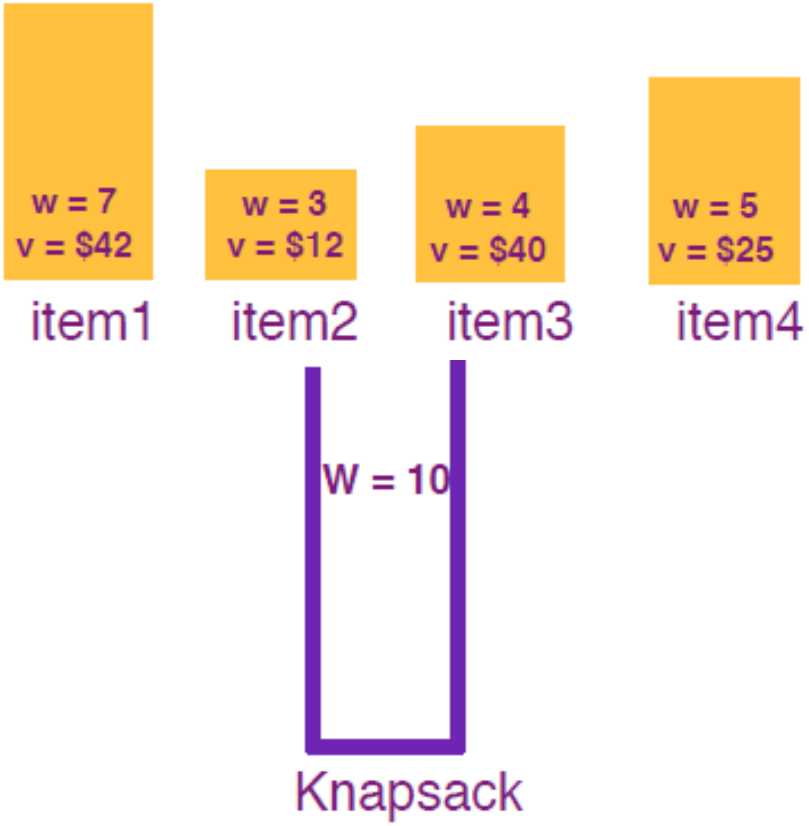
# Knapsack Problem

- Given $n$ items of known weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ and a knapsack of capacity $W$, find the most valuable subset of the items that fit into the knapsack.

- Consider all the subsets of the set of $n$ items given,
  - Computing the total weight of each subset in order to identify feasible subsets (the ones with the total not exceeding the knapsack's capacity).
  - Finding a subset of the largest value among them.

# Knapsack – how to work

| Subset | Total weight | Total value |
|---|---|---|
| Null | 0 | $0 |
| {1} | 7 | $42 |
| {2} | 3 | $12 |
| {3} | 4 | $40 |
| {4} | 5 | $25 |
| {1,2} | 10 | $36 |
| {1,3} | 11 | not feasible |
| {1,4} | 12 | not feasible |
| {2,3} | 7 | $52 |
| {2,4} | 8 | $37 |
| {3,4} | 9 | $65 |
| {1,2,3} | 14 | not feasible |
| {1,2,4} | 15 | not feasible |
| {1,3,4} | 16 | not feasible |
| {2,3,4} | 12 | not feasible |
| {1,2,3,4} | 19 | not feasible |



# Brute Force Method

- Enumerates every possible packing configuration
- Choose the best solution
- Optimality is ensured
- Extremely costly in time, for large n

# 효율성

- Since there are n items, there are $2^n$ possible combinations of items.
- We go through all combinations and find the one with the maximum value and with total weight less or equal to m.
- Running time will be $O(2^n)$

# Knapsack Problem을 위한 전략

- 다른 전략으로 해결법
  - There are two versions of the problem:
    1. “Fractional” knapsack problem.
    2. “0/1” knapsack problem.
  - 1 Items are divisible: you can take any fraction of an item. Solved with a **greedy algorithm**.
  - 2 Item are indivisible; you either take an item or not. Solved with **dynamic programming**.

# Job Assignment Problem

# Job Assignment Problem

- There are *n* people who need to be assigned to *n* jobs, one person per job. The cost of assigning person *i* to job *j* is C[*i*,*j*].
- Find an assignment that minimizes the total cost.
- Select one element in each row so that all selected elements are in different columns and the total sum of the selected elements is the smallest possible.

# Job Assignment Problem

| | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| Person 1 | **9** | **2** | **7** | **8** |
| Person 2 | **6** | **4** | **3** | **7** |
| Person 3 | **5** | **8** | **1** | **8** |
| Person 4 | **7** | **6** | **9** | **4** |

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

| Assignment (col.#s) | Total Cost |
|---|---|
| 1, 2, 3, 4 | 9+4+1+4=18 |
| 1, 2, 4, 3 | 9+4+8+9=30 |
| 1, 3, 2, 4 | 9+3+8+4=24 |
| 1, 3, 4, 2 | 9+3+8+6=26 |
| 1, 4, 2, 3 | 9+7+8+9=33 |
| 1, 4, 3, 2 | 9+7+1+6=23 |
| | etc. |

# What if 100 persons and 100 Jobs

- Since the solution is a permutation of the n jobs, its complexity is O(n!).

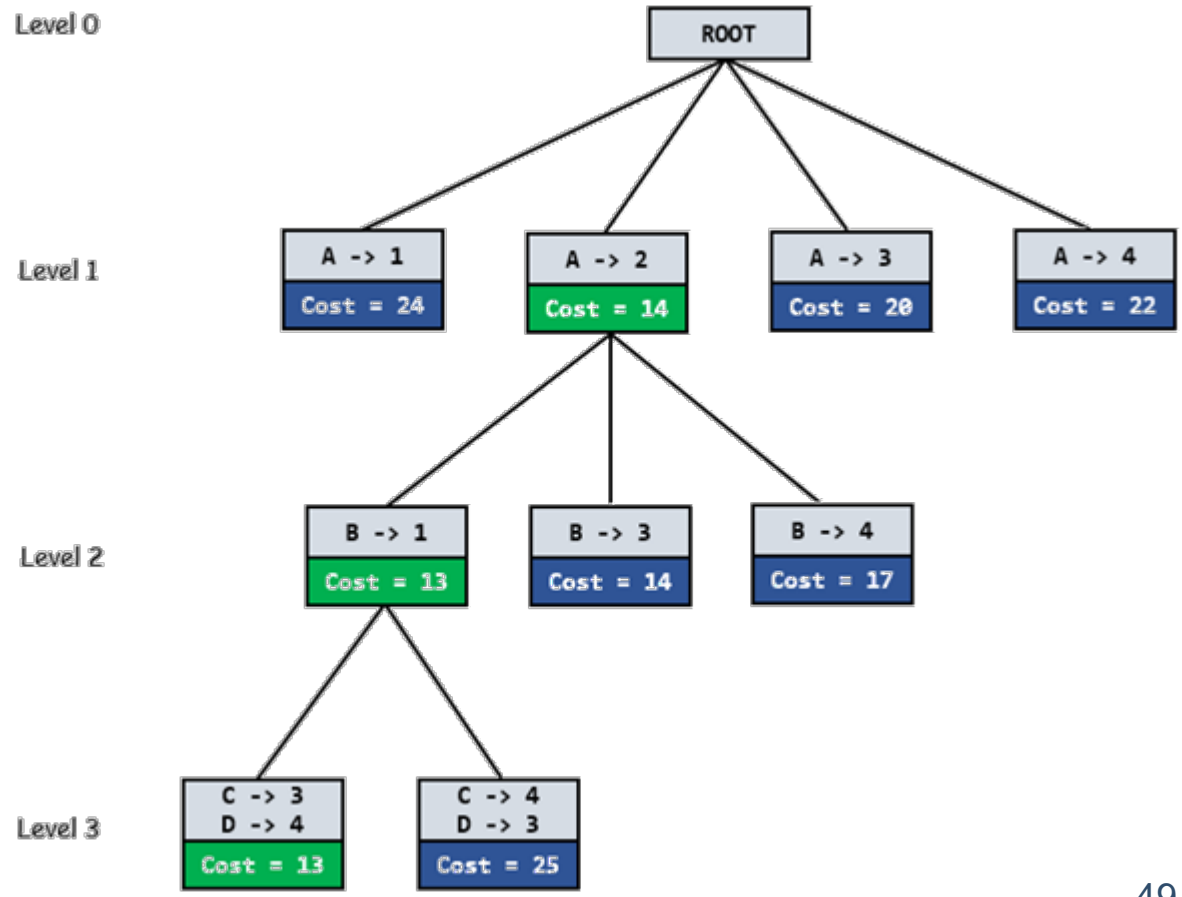
```
>>> def factorial_for(n):
    ret = 1
    for i in range(1, n+1):
        ret *= i
    return ret

>>> factorial_for(100)
93326215443944152681699238856266700490715968264381621468
592963895217599993229915608941463976156518286253697920827
223758251185210916864000000000000000000000000
>>>
```

# Branch and Bound

- 처리 과정

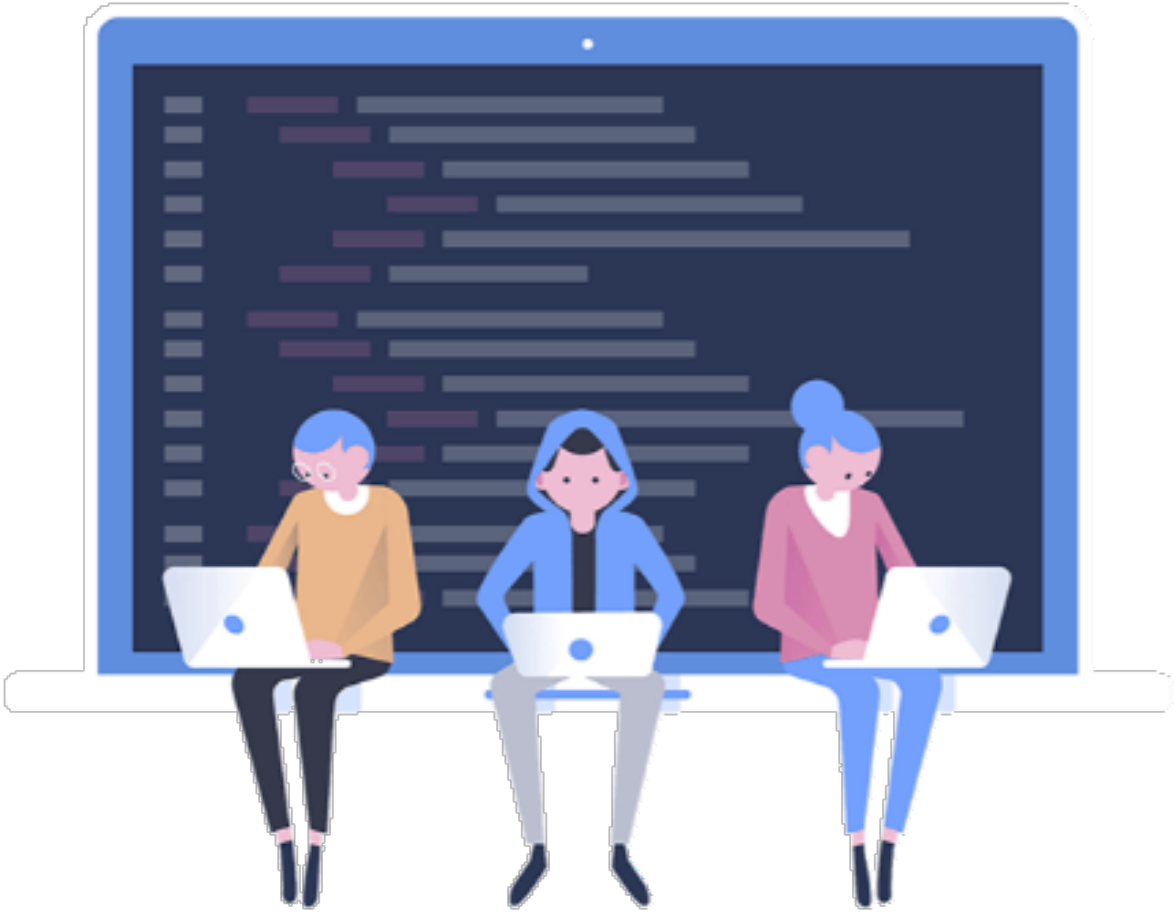
| | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| A | 9 | 2 | 7 | 8 |
| B | 6 | 4 | 3 | 7 |
| C | 5 | 8 | 1 | 8 |
| D | 7 | 6 | 9 | 4 |



# 결과

- Brute Force는 모든 경우를 다 계산 후 optimal solution

| | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| A | 9 | 2 | 7 | 8 |
| B | 6 | 4 | 3 | 7 |
| C | 5 | 8 | 1 | 8 |
| D | 7 | 6 | 9 | 4 |

An example job assignment problem. Green values show optimal job assignment that is A-Job4, B-Job1 C-Job3 and D-Job4

# 최고의 프로그래머가 되는 그날까지!!